

Quick Start Guide — Command Station

Get a DCC command station running in minutes

TI MSPM0G3507 LaunchPad + Code Composer Studio / Theia edition. OpenDccCLib itself runs on any microcontroller with a C compiler; this guide covers one concrete path.

Generated 2026-09-25 from the repository sources by documentation/guides/build_pdfs.py

Table of Contents

1. What is DCC?	3
1.1 Key Terms	3
2. What You Need	4
2.1 Hardware	4
2.2 Signal Pins	4
2.3 Software	4
3. Project Setup	5
4. Understanding dcc_user_config.h	6
5. Building and Flashing	7
5.1 What Happens at Power-On	7
6. Using the UART Command Interface	8
6.1 Quick Test	10
7. What's Next	11

1. What is DCC?

DCC (Digital Command Control) is the NMRA standard for controlling model railroad locomotives and accessories. A command station generates the DCC signal, a modulated square wave on the rails that carries addressed speed, function and programming commands to decoders in locomotives and accessories.

OpenDccCLib is a portable C library that implements the protocol: bit encoding, packet scheduling, service-mode programming and RailCom, so you can concentrate on what your command station does.

1.1 Key Terms

- **Command station:** generates DCC packets on the track.
- **Decoder:** receives and acts on DCC commands, in a locomotive or an accessory.
- **Packet:** one command, address plus instruction plus error check, framed by a preamble and an end bit.
- **Service mode:** the programming-track mode for reading and writing decoder Configuration Variables (CVs).
- **RailCom:** the bidirectional channel in which decoders answer during a short cutout after each packet.
- **Scheduler:** the library component that manages concurrent packets, priorities, repeats and auto-refresh.

2. What You Need

2.1 Hardware

- TI MSPM0G3507 LaunchPad (LP-MSPM0G3507)
- An H-bridge driver board (L298N, IBT-2 or similar) and a 12–18 V supply for track power
- A DCC locomotive with a decoder, or a second LaunchPad running the decoder example
- A USB cable, for programming and the UART terminal

2.2 Signal Pins

The pin assignments come from the project's SysConfig file.

LaunchPad pin	Function
PB1	Main-track DCC signal out (DCC_SIGNAL); to the H-bridge input, or to decoder PB1
PB4	Service-track DCC signal out; to decoder PB4
PB12	ACK current-sense in; from decoder PB12
PB17	Track-select out; to decoder PB17. High while the service track is powered or service mode is active (the decoder then listens on PB4), low otherwise
PB2	DCC mirror out (scope aid)
PB3	Debug pulse (scope aid)
PA15	ISR timing pulse (scope aid)
PB22, PB26, PB27	User LEDs; PB22 is the heartbeat
PA10 / PA11	UART TX / RX to the on-board debug probe, 230400 8N1

Two-board setup: CS PB1 to decoder PB1, CS PB4 to decoder PB4, decoder PB12 to CS PB12, CS PB17 to decoder PB17, and GND to GND.

2.3 Software

- TI Code Composer Studio or CCS Theia, with SysConfig (bundled)
- A serial terminal at **230400 baud**, 8N1
- The OpenDccCLib source tree

3. Project Setup

The ready-to-run example is at `applications/ti_theia/mspm03507_launchpad/command_station/`.

1. In CCS, choose File > Import > CCS Projects and point it at that folder.
2. The project reaches the library through the `dcc_lib` symlink to `src/dcc/`; nothing is copied.
3. Build once so SysConfig generates `ti_msp_dl_config.c/h` into `Debug/`.

```
command_station/  
  command_station.c          <- main entry: config struct, ISRs, main loop  
  dcc_user_config.h          <- feature flags and sizes  
  uart_command_parser.c/h    <- UART command-line interface  
  application_callbacks/  
    callbacks_dcc.c/h        <- your application logic  
  application_drivers/  
    ti_driverlib_dcc_driver.c/h <- timers, GPIO, current sense  
    ti_driverlib_uart_driver.c/h <- UART I/O  
  dcc_lib/                   <- the library (symlink; do not edit)  
  Debug/ti_msp_dl_config.c/h <- SysConfig-generated init
```

4. Understanding dcc_user_config.h

This file tells the library which role to compile and how much memory to reserve. Every constant except `USER_DEFINED_DCC_ACK_DROPOUT_TOLERANCE_US` (default 116 μ s) is mandatory and is checked at compile time.

```
#define DCC_COMPILE_COMMAND_STATION
#define DCC_COMPILE_RAILCOM                // cutout + receive; comment out to strip all RailCom code
#define DCC_COMPILE_SERVICE_MODE_DIRECT    // and PAGED, REGISTER, ADDRESS
#define DCC_COMPILE_SERVICE_MODE_TASK_DIRECT // ... plus the TASK_* orchestrators and TASK_DETECT

#define USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT 24 // concurrent packets
#define USER_DEFINED_DCC_PREAMBLE_BITS_OPS    18 // >= 16 with RailCom; 18 or more avoids a #warning
#define USER_DEFINED_DCC_MAX_LOCOS            10 // demo loco table
#define USER_DEFINED_DCC_RAILCOM_BUFFER_DEPTH 4
#define USER_DEFINED_DCC_SERVICE_MODE_RETRIES 3
#define USER_DEFINED_DCC_ACK_THRESHOLD_MA     60
#define USER_DEFINED_DCC_ACK_MIN_DURATION_US  5000
#define USER_DEFINED_DCC_ACK_MAX_DURATION_US  7000
#define USER_DEFINED_DCC_ACK_DROPOUT_TOLERANCE_US 116
```

The four primitive flags (`DCC_COMPILE_SERVICE_MODE_DIRECT`, `_PAGED`, `_REGISTER`, `_ADDRESS`) require `DCC_COMPILE_COMMAND_STATION`; the compiler stops with a `#error` otherwise. The `TASK_*` flags are not cross-checked. `DCC_COMPILE_RAILCOM` needs a role flag.

5. Building and Flashing

1. Build the project (Project > Build, or Ctrl+B).
2. Connect the LaunchPad over USB.
3. Flash and run (Run > Debug, or F11).
4. Open a terminal at 230400 baud on the LaunchPad's serial port.
5. The banner reads DCC Command Station - MSPM0G3507 LaunchPad.

5.1 What Happens at Power-On

`main()` initializes the drivers, hands the `dcc_config_t` struct to `DccConfig_initialize()`, then loops. Track power is off until you type `POWER ON`.

```
while (1) {  
    DccConfig_run();           // scheduling, callbacks  
    TI_UartDriver_echo_process(); // echo typed characters  
    UartCommandParser_process(); // parse and execute commands  
}
```

6. Using the UART Command Interface

Type `HELP` for the full list. Commands answer `OK: ...` or `ERR: ...`; `STATUS` prints a `STATUS: line` and `HELP` prints the list. A service-mode operation answers `OK: at once` and `SVC RESULT: ...` (or `SVC DETECT: ...`) when it finishes.

Com mand	Description			
`POWER ON	OFF`	Track power		
`SPEED <addr> <speed> <FWD	REV> [14	28	128]`	Speed and direction. With REFRESH ON (the default) the packet is auto-refreshed: three prompt sends on consecutive packet cycles, then a keep-alive every 60 packet cycles and never later than 120. With REFRESH OFF it is a one-shot sent twice
ESTOP [addr]	Emergency stop, one loco or broadcast. An addressed ESTOP replaces that loco's speed refresh slot with a one-shot, so issue SPEED again afterwards			
STOP	Broadcast controlled stop			
`FUNCTION <addr> <0 -68> <ON	OFF>`	Function on on or off; auto-r efresh ed the same way		

Com mand	Description		
`ACC <board> <pair> <ON	OFF> / ACCE <addr> <aspect> / NOP <addr> [E]`	Basic access sory, e xtende d acce ssory, access sory NOP	
ACC CV ... / ACCE CV ...	Accessory o perations-mo de CV write, verify, bit		
`CV W RITE	VERIFY <addr> <cv> <value> / CV BIT <addr> <cv> <bit> <0	1>` Loco o peratio ns-mo de CV access s	
`CON SIST <addr> > SET <ca> [NORM AL	REVERSE] / CONSIST <addr> CLEAR`	Advan ced co nsist	
BSS, BSL, A NALOG	Binary state short and long, analog function		
SYSTI ME <ms> / MTIME ... / MDATE <d> <m> <y>	Broadcast time and date		
SVC ENTER / SVC EXIT / SVC D ETECT	Service mode on the programming track		

Com mand	Description								
`SVC DIRE CT W RITE	READ	BITW	BITR ... / SVC P AGED WRIT E	READ ... / SVC REG WRITE	READ [MOBI LE	ACC]	RESE T / SVC ADDR WRIT E	READ ,	Read and write CVs in each mode; HELP lists the ar gumen ts
`REFR ESH ON	OFF / CLEAR / RESET`	Auto-r efresh policy, clear the sc hedule r, broa dcast reset							
STATU S / HELP	Status line, command list								

6.1 Quick Test

```

> POWER ON
OK: track power ON
> SPEED 3 50 FWD 128
OK: SPEED addr=3 speed=50 dir=FWD mode=128
> FUNC 3 0 ON
OK: FUNC addr=3 F0=ON
> ESTOP
OK: ESTOP broadcast
> POWER OFF
OK: track power OFF

```

A one-shot command such as CV WRITE goes out the number of times the standard requires (twice for a CV write); the library's packet builders carry those counts.

7. What's Next

- **RailCom detection.** Wire a RailCom detector's UART output to a 250 kbaud UART, fill a `dcc_railcom_hw_t`, point `main_track.railcom` at it and provide `railcom_timer_start/stop`. The library runs the cutout, gates the receiver to the reply windows, and decodes each channel from the bytes your `uart_read` tags with that channel (`DCC_RAILCOM_CH1 / DCC_RAILCOM_CH2`). It delivers each channel's result (a datagram, ACK, NACK or an error code in `datagram->result`) to `on_railcom_datagram_result`, tagged with the loco address of the preceding packet (datagrams after accessory, broadcast or idle packets carry address 0). The receive hooks in `dcc_config.h` state the gating contract.
- **Current-sense protection.** The library samples only `service_track.current_sense_read`, for ACK detection. Main-track overcurrent protection belongs to your application or the H-bridge board; `main_track.current_sense_read` is not read by the library.
- **Another MCU.** Copy the project and rewrite the two driver files for your timers and GPIO. The library never touches hardware directly.
- **Developer Guide.** The companion Developer Guide, Command Station, explains the config struct, scheduler, bit encoder, service mode and RailCom in depth, and the hardware-in-the-loop bench under `test/compliance/` that proves the wire.